

3.1. Configuración del entorno de desarrollo y herramientas



DESARROLLO DE APLICACIONES WEB MULTIPLATAFORMA NIVEL 3.

UNIDAD 3:

INTRODUCCION A FLUTTER

Contenido

Introducción.....	3
Importancia y papel que juega.....	3
Ventajas de usar Flutter para desarrollo multiplataforma.....	3
Requisitos previos para instalar y trabajar con Flutter	4
Consideraciones adicionales	5
Instalación de Flutter	6
Actualización de Flutter.....	7
Consideraciones adicionales:	7
Herramientas de desarrollo para Flutter	8
Android Studio como IDE principal.....	8
VSCode como alternativa.....	9
Emuladores, simuladores y depuradores.....	10
Simuladores y emuladores para probar aplicaciones	10
Simuladores:.....	10
Emuladores:	11
Elección entre simuladores y emuladores:	11
Depuración y análisis de código	11
Depuración:	11
Análisis de código:	12
Integración de emuladores, depuradores y análisis de código	12
Recursos adicionales.....	13

Introducción

Flutter es un SDK (Software Development Kit) de código abierto creado por Google para desarrollar aplicaciones móviles multiplataforma. Esto significa que con Flutter puedes crear una única base de código y usarla para generar aplicaciones que se ejecuten de forma nativa en dispositivos Android, iOS, web, escritorio (Windows, macOS, Linux) y hasta en Fuchsia (el futuro sistema operativo de Google).

Importancia y papel que juega

Flutter está ganando rápidamente popularidad entre los desarrolladores debido a sus múltiples ventajas, que la convierten en una herramienta poderosa y versátil para el desarrollo de aplicaciones móviles. Entre los factores que contribuyen a su importancia destacan:

- Desarrollo multiplataforma: Permite crear aplicaciones con un solo código base, lo que ahorra tiempo, esfuerzo y recursos en comparación con el desarrollo nativo para cada plataforma.
- Rendimiento nativo: Las aplicaciones Flutter se compilan en código nativo para cada plataforma, lo que garantiza un rendimiento fluido y una experiencia de usuario óptima.
- Interfaz de usuario rica: Flutter ofrece una amplia gama de widgets y herramientas para crear interfaces de usuario hermosas y con gran capacidad de respuesta.
- Facilidad de aprendizaje: Dart, el lenguaje de programación utilizado en Flutter, es relativamente fácil de aprender, especialmente para aquellos que ya tienen experiencia en otros lenguajes de programación.
- Comunidad grande y en crecimiento: Flutter cuenta con una comunidad de desarrolladores activa y en constante crecimiento que proporciona soporte, comparte recursos y contribuye al desarrollo del framework.

Ventajas de usar Flutter para desarrollo multiplataforma

Las ventajas de usar Flutter para desarrollo multiplataforma son numerosas:

- Reducción del tiempo y costo de desarrollo: Al crear una sola base de código para múltiples plataformas, se reduce significativamente el

tiempo y el costo de desarrollo en comparación con el desarrollo nativo para cada plataforma.

- Mayor alcance de mercado: Las aplicaciones Flutter pueden llegar a una audiencia más amplia al ser compatibles con una amplia gama de dispositivos y sistemas operativos.
- Experiencia de usuario consistente: Las aplicaciones Flutter ofrecen una experiencia de usuario consistente en todas las plataformas, lo que garantiza una alta calidad y satisfacción del usuario.
- Desarrollo más rápido: Flutter ofrece herramientas y características que agilizan el proceso de desarrollo, como la recarga en caliente y la depuración integrada.
- Facilidad de mantenimiento: Al tener una sola base de código, el mantenimiento y la actualización de las aplicaciones Flutter son más sencillos y económicos.

Requisitos previos para instalar y trabajar con Flutter

Esos son los elementos básicos que necesitas para construir un entorno de desarrollo para trabajar con Flutter y comenzar a crear aplicaciones multiplataforma.

- Ordenador:
Sistema operativo: Windows, macOS o Linux.
Memoria RAM: Se recomienda un mínimo de 8 GB para un rendimiento fluido.
Almacenamiento: Se recomiendan 64 GB de almacenamiento libre para alojar el IDE, el SDK de Flutter y los proyectos de desarrollo.
- Entorno de desarrollo:
Android Studio: El IDE oficial para Flutter, ofrece herramientas integradas para desarrollo, depuración y pruebas o VSCode: Un editor de código alternativo popular, con extensiones disponibles para Flutter.
- SDK de Flutter:
Descargable e instalable desde el sitio web oficial de Flutter:
<https://docs.flutter.dev/get-started/install>
Contiene las herramientas y bibliotecas necesarias para desarrollar aplicaciones Flutter.
En el epígrafe siguiente veremos con más detalle este proceso

- Dispositivo móvil:
Android o iOS: Necesario para probar las aplicaciones desarrolladas en dispositivos reales.
Puedes utilizar emuladores o simuladores si no tienes un dispositivo físico disponible.
- Conocimientos previos:
Experiencia en programación: Se recomienda para comprender mejor los conceptos de Flutter y el desarrollo de aplicaciones.
Familiaridad con lenguajes de programación: Conceptos básicos de Java, Kotlin o Swift pueden ser útiles, ya que Dart, el lenguaje de Flutter, comparte algunas similitudes.

Consideraciones adicionales:

Es recomendable tener una cuenta de Google para acceder a recursos adicionales y herramientas de desarrollo de Flutter.

Unirse a la comunidad de Flutter puede ser útil para obtener ayuda, compartir conocimientos y estar actualizado con las últimas novedades.

Instalación de Flutter



Para descargar e instalar el SDK de Flutter debes seguir los siguientes pasos:

1. Descargar el SDK de Flutter:
 - a. Accede al sitio web oficial de Flutter: <https://docs.flutter.dev/get-started/install>
 - b. Haz clic en el botón "Descargar SDK de Flutter".
 - c. Elige el sistema operativo de tu ordenador (Windows, macOS o Linux).
 - d. Descarga el archivo de instalación correspondiente.
2. Instalar el SDK de Flutter:
 - a. Una vez descargado el archivo de instalación, ejecútalo.
 - b. Sigue las instrucciones del asistente de instalación.
 - c. Acepta los términos de licencia y elige la ruta de instalación.
 - d. Asegúrate de agregar la ruta de instalación de Flutter a tu variable de entorno PATH.
3. Configuración del entorno de desarrollo (Android Studio, VSCode)

Para trabajar con Android Studio necesitarás:

 - a. Instalar Android Studio:
 - i. Descarga e instala Android Studio desde el sitio web oficial: <https://developer.android.com/studio>
 - ii. Sigue las instrucciones del asistente de instalación.
 - b. Configurar el plugin de Flutter:
 - i. Abre Android Studio.
 - ii. Ve a File > Settings > Plugins.
 - iii. Busca "Flutter" en la barra de búsqueda.
 - iv. Instala el plugin "Flutter".

- v. Reinicia Android Studio.

Si prefieres trabajar con VSCode:

- a. Instalar VSCode
 - i. Descarga e instala VSCode desde el sitio web oficial: <https://code.visualstudio.com/download>
 - ii. Sigue las instrucciones del asistente de instalación.
 - b. Instalar la extensión de Flutter:
 - i. Abre VSCode.
 - ii. Ve a Extensions.
 - iii. Busca "Flutter" en la barra de búsqueda.
 - iv. Instala la extensión "Dart and Flutter".
 - v. Sigue las instrucciones para configurar la extensión.
4. El paso final común para los dos es la verificación de la instalación:
- a. Abre una terminal o ventana de comandos.
 - b. Ejecuta el comando flutter doctor.

Este comando te proporcionará información sobre la configuración de tu entorno de desarrollo y te indicará si hay algún problema que deba solucionarse.

Actualización de Flutter

Para actualizar Flutter a la última versión, puedes ejecutar el siguiente comando en una terminal o ventana de comandos:

```
flutter upgrade
```

Este comando descargará e instalará las últimas actualizaciones del SDK de Flutter.

Consideraciones adicionales:

Es recomendable reiniciar tu IDE o editor de código después de instalar o actualizar Flutter.

Puedes consultar la documentación oficial de Flutter para obtener más información sobre la instalación y configuración: <https://docs.flutter.dev/get-started/install>

Herramientas de desarrollo para Flutter



En este módulo profundizaremos en las principales herramientas de desarrollo que te ayudarán a crear aplicaciones Flutter de forma eficiente y eficaz.

Ya vimos que son dos las opciones:

- Android Studio como IDE principal
- VSCode como alternativa

Profundicemos en cada una de ellas:

Android Studio como IDE principal

Android Studio es el IDE oficial para el desarrollo de Flutter, ofreciendo un entorno completo con las siguientes características:

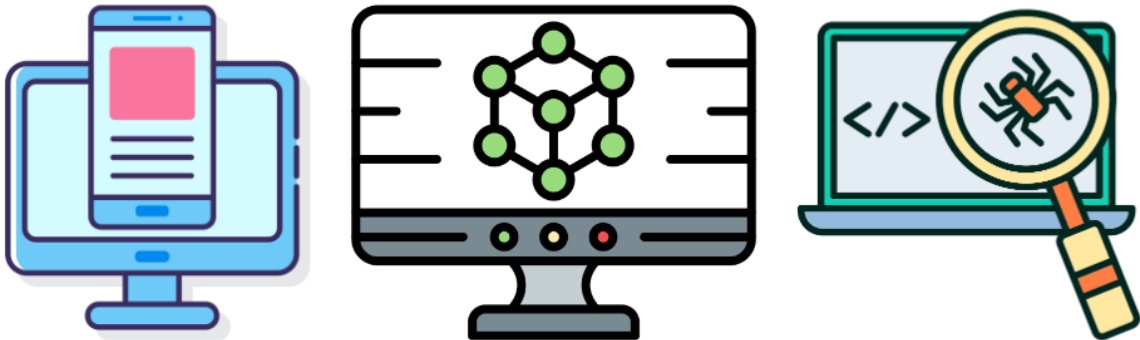
- Editor de código integrado: Permite escribir, editar y navegar por el código Dart de tus aplicaciones Flutter.
- Complemento de Flutter: Brinda herramientas específicas para el desarrollo de Flutter, como el depurador de Flutter, el emulador de Flutter y el widget inspector.
- Soporte para desarrollo multiplataforma: Permite crear y probar aplicaciones Flutter para diferentes plataformas (Android, iOS, web, escritorio).
- Integración con herramientas de desarrollo de Android: Útil si estás familiarizado con el desarrollo nativo de Android.

VSCode como alternativa

VSCode es un editor de código ligero y personalizable que también puede utilizarse para desarrollar aplicaciones Flutter. Ofrece las siguientes ventajas:

- Ligero y rápido: Consume menos recursos del sistema que Android Studio.
- Altamente personalizable: Permite instalar extensiones y plugins para ampliar sus funcionalidades.
- Soporte multiplataforma: Funciona en Windows, macOS y Linux.
- Popularidad: Una gran comunidad de desarrolladores lo utilizan, lo que facilita encontrar ayuda y recursos.

Emuladores, simuladores y depuradores



En el desarrollo de aplicaciones Flutter, los emuladores, simuladores y depuradores son herramientas fundamentales que te permiten probar, depurar y optimizar tu código para garantizar una experiencia de usuario fluida y libre de errores.

Simuladores y emuladores para probar aplicaciones

Veamos cada uno de ellos:

Simuladores:

Son programas de software que recrean el comportamiento de un dispositivo real, pero sin utilizar su hardware físico.

Ventajas:

- Son más rápidos y ligeros que los emuladores.
- Consumen menos recursos del sistema.
- Permiten realizar pruebas básicas de la funcionalidad de la aplicación.

Desventajas:

- No replican exactamente el comportamiento de un dispositivo real.
- Pueden presentar limitaciones al probar ciertas características específicas del dispositivo.

Emuladores:

Son programas de software que imitan el hardware y software de un dispositivo real, creando una virtualización completa del dispositivo.

Ventajas:

- Ofrecen una emulación más precisa del dispositivo real.
- Permiten probar todas las características y funcionalidades de la aplicación.

Desventajas:

- Son más lentos y consumen más recursos del sistema que los simuladores.
- Pueden requerir más tiempo para configurarse y ejecutarse.

Elección entre simuladores y emuladores:

- Para pruebas básicas y desarrollo inicial: Los simuladores suelen ser suficientes.
- Para pruebas más completas y precisas: Se recomienda utilizar emuladores.
- Considerar el dispositivo objetivo: Utilizar el emulador del dispositivo específico en el que se ejecutará la aplicación.

Depuración y análisis de código

Con más detalle:

Depuración:

Es el proceso de identificar y corregir errores en el código de una aplicación.

Herramientas de depuración:

- Depurador de Flutter: Integrado en Android Studio y VSCode, permite establecer puntos de interrupción, examinar variables y ejecutar código paso a paso.
- Consola de depuración: Muestra mensajes de error y advertencias durante la ejecución de la aplicación.

Beneficios de la depuración:

- Permite identificar y corregir errores de manera eficiente.
- Ayuda a mejorar la calidad y el rendimiento del código.
- Facilita la comprensión del funcionamiento interno de la aplicación.

Análisis de código:

Es el proceso de examinar el código para identificar posibles problemas de estilo, rendimiento, seguridad y mantenibilidad.

Herramientas de análisis de código:

- Analizadores estáticos: Detectan errores potenciales en el código sin necesidad de ejecutarlo.
- Linters: Ayudan a mantener un estilo de código consistente y evitar malas prácticas de programación.

Beneficios del análisis de código:

- Mejora la calidad y legibilidad del código.
- Reduce la probabilidad de errores en el futuro.
- Facilita la colaboración entre desarrolladores.

Integración de emuladores, depuradores y análisis de código

Las herramientas de emulación, depuración y análisis de código se integran perfectamente en el entorno de desarrollo de Flutter, permitiendo un flujo de trabajo eficiente:

- Desarrollo y pruebas básicas con simuladores: Utilizar simuladores para realizar pruebas iniciales y verificar la funcionalidad básica de la aplicación.
- Depuración avanzada con emuladores: Cuando se detectan errores, utilizar emuladores para depurar el código paso a paso y localizar la causa del problema.
- Análisis de código para mejorar la calidad: Antes de la entrega final, realizar un análisis de código para identificar y corregir posibles problemas de estilo, rendimiento, seguridad y mantenibilidad.

Recursos adicionales

Documentación oficial de Flutter: <https://docs.flutter.dev/>

Tutoriales de Flutter: <https://docs.flutter.dev/get-started/codelab>

Comunidad de Flutter: <https://flutter.dev/community>

Foros de Flutter: <https://meta.discourse.org/t/dart-flutter-code-block-support/250122>

¡Enhorabuena por completar esta unidad didáctica sobre Configuración del entorno de desarrollo y herramientas para Flutter!

En este módulo hemos recorrido los pasos esenciales para comenzar tu viaje en el apasionante mundo del desarrollo de aplicaciones Flutter. Sin duda, has adquirido una base sólida para empezar a adentrarte en la creación de aplicaciones multiplataforma con Flutter.

Recuerda que este es solo el inicio de un camino lleno de aprendizaje y crecimiento. A medida que avances, descubrirás nuevas herramientas, técnicas y conceptos que te permitirán crear aplicaciones cada vez más complejas y sofisticadas.

No dudes en seguir explorando, experimentando y poniendo en práctica lo aprendido. La comunidad Flutter es un gran recurso para encontrar ayuda, inspiración y apoyo.

¡Mucha suerte en tu camino como desarrollador Flutter!

¡Hasta la próxima!